

Day 19: コメント編集・削除を実装しよう

今日のゴール

投稿したコメントを編集・削除できる機能を実装します。自分のコメントだけに編集・削除ボタンを表示します。

【スクリーンショット: コメント編集モード】

なぜこれを作るのか?

誤字や情報の変更に対応するための機能です。コメント編集は消しゴムと修正ペンのようなもの。ノートに書いた文章を消したり書き直したりできるように、一度投稿したコメントも後から修正できると便利です。

実装ステップ一覧

ステップ	作業内容	所要時間
Step 1	編集・削除ボタン表示	10分
Step 2	編集モード切り替え	15分
Step 3	編集API呼び出し	10分
Step 4	削除API呼び出し	10分

合計時間: 約45分

Step 1: 編集・削除ボタン表示 (10分)

実装:

```

// filepath: src/components/task/TaskComments.tsx
import { useSession } from 'next-auth/react';
import { IconButton } from '@mui/material';
import EditIcon from '@mui/icons-material/Edit';
import DeleteIcon from '@mui/icons-material/Delete';

export function TaskComments({ taskId }: TaskCommentsProp) {
  const { data: session } = useSession();
  const { data: comments } = api.comment.getByTask.useQuery(taskId)

  return (
    <Box sx={{ mt: 3 }}>
      {comments?.map((comment) => (
        <Paper key={comment.id} sx={{ p: 2, mb: 2 }}>
          <Box sx={{ display: 'flex', justifyContent: 'space-between' }}>
            <Box sx={{ display: 'flex', alignItems: 'center' }}>
              <Avatar sx={{ width: 32, height: 32, mr: 1 }} src={comment.user.avatar} />
              <Typography variant="body2" fontWeight="bold">{comment.user.name}</Typography>
            </Box>
            <Box>
              <Typography variant="body2">{comment.content}</Typography>
              {session?.user?.id === comment.userId && (
                <Box>
                  <IconButton size="small">
                    <EditIcon fontSize="small" />
                  </IconButton>
                  <IconButton size="small" color="error">
                    <DeleteIcon fontSize="small" />
                  </IconButton>
                </Box>
              )}
            </Box>
          </Paper>
        )}
    </Box>
  )
}

```

```
        </Paper>
      )})
    </Box>
  );
}
```

確認ポイント: 自分のコメントにのみ編集・削除ボタンが表示される

Step 2: 編集モード切り替え（15分）

実装:

```

// filepath: src/components/task/TaskComments.tsx
import { useState } from 'react';
import { TextField, Button } from '@mui/material';

export function TaskComments({ taskId }: TaskCommentsProp) {
  const [editingId, setEditingId] = useState<string | null>(null);
  const [editContent, setEditContent] = useState('');

  const handleStartEdit = (commentId: string, content: string) => {
    setEditingId(commentId);
    setEditContent(content);
  };

  const handleCancelEdit = () => {
    setEditingId(null);
    setEditContent('');
  };

  return (
    <Box sx={{ mt: 3 }}>
      {comments?.map((comment) => (
        <Paper key={comment.id} sx={{ p: 2, mb: 2 }}>
          <Box sx={{ display: 'flex', justifyContent: 'space-between' }}>
            <Box sx={{ display: 'flex', alignItems: 'center' }}>
              <Avatar sx={{ width: 32, height: 32, mr: 1 }} alt={comment.user.name?.[0]} />
              <Typography variant="body2" fontWeight="bold">
                {comment.user.name}
              </Typography>
            </Box>
            <Box>
              {session?.user?.id === comment.userId && !editingId ? (
                <IconButton alt="edit" size="small" />
              ) : (
                <IconButton alt="delete" size="small" />
              )}
            </Box>
          </Box>
          <TextField
            value={editContent}
            onChange={e => setEditContent(e.target.value)}
            style={{ width: 100%; margin-top: 5px; border: 1px solid #ccc; border-radius: 5px; padding: 5px; }}
          />
        </Paper>
      ))}
    </Box>
  );
}

```

```

        onClick={() => handleStartEdit(comment.
    >
        <EditIcon fontSize="small" />
    </IconButton>
</Box>
    )}
</Box>

{editingId === comment.id ? (
    <Box>
        <TextField
            fullWidth
            multiline
            rows={3}
            value={editContent}
            onChange={(e) => setEditContent(e.target.
            sx={{ mb: 1 }}
        />
        <Box sx={{ display: 'flex', gap: 1 }}>
            <Button size="small" variant="contained">
                保存
            </Button>
            <Button size="small" onClick={handleCance
                キャンセル
            </Button>
        </Box>
    </Box>
    ) : (
        <Typography variant="body2">{comment.content}
    )}
</Paper>
    )}
</Box>
);
}

```

確認ポイント: 編集ボタンで編集モードに切り替わる

Step 3: 編集API呼び出し (10分)

実装:

```
// filepath: src/components/task/TaskComments.tsx
export function TaskComments({ taskId }: TaskCommentsProps) {
  const utils = api.useUtils();

  const updateCommentMutation = api.comment.update.useMutation({
    onSuccess: () => {
      setEditingId(null);
      setEditContent('');
      utils.comment.getByTask.invalidate({ taskId });
    },
  });

  const handleSaveEdit = (commentId: string) => {
    if (!editContent.trim()) return;

    updateCommentMutation.mutate({
      id: commentId,
      content: editContent.trim(),
    });
  };

  return (
    // UI は同じ
    <Button
      size="small"
      variant="contained"
      onClick={() => handleSaveEdit(comment.id)}
      disabled={updateCommentMutation.isPending}
    >
      {updateCommentMutation.isPending ? '保存中...' : '保存'}
    </Button>
  );
}
```

確認ポイント: コメントが更新される

Step 4: 削除API呼び出し（10分）

実装:

```
// filepath: src/components/task/TaskComments.tsx
export function TaskComments({ taskId }: TaskCommentsProp) {
  const deleteCommentMutation = api.comment.delete.useMutation({
    onSuccess: () => {
      utils.comment.getByTask.invalidate({ taskId });
    },
  });

  const handleDelete = (commentId: string) => {
    if (!confirm('このコメントを削除しますか?')) return;

    deleteCommentMutation.mutate({ id: commentId });
  };

  return (
    <Box sx={{ mt: 3 }}>
      {comments?.map((comment) => (
        <Paper key={comment.id} sx={{ p: 2, mb: 2 }}>
          {session?.user?.id === comment.userId && !editingCommentId === comment.id ? (
            <Box>
              <IconButton
                size="small"
                onClick={() => handleStartEdit(comment.id)}
              >
                <EditIcon fontSize="small" />
              </IconButton>
              <IconButton
                size="small"
                color="error"
                onClick={() => handleDelete(comment.id)}
              >
                <DeleteIcon fontSize="small" />
              </IconButton>
            </Box>
          ) : null}
        </Paper>
      ))}
    </Box>
  );
}
```

```
    )})  
  </Box>  
  );  
}
```

確認ポイント: 確認ダイアログが表示され、コメントが削除される

学んだこと

- **IconButton:** アイコンだけのボタン（省スペース）
- **条件付きレンダリング:** 三項演算子で編集モードと表示モードを切り替え
- **権限チェック:** `session.user.id === comment.userId` で本人確認
- **状態管理:** `editingId` で「どのコメントを編集中か」を管理
- **confirm関数:** 削除前の確認ダイアログ

今日のまとめ

- ☐ 編集・削除ボタンを表示できた
- ☐ 編集モードに切り替えられるようにした
- ☐ 編集APIを呼び出してコメントを更新できた
- ☐ 削除APIを呼び出してコメントを削除できた

⚠ つまづきポイント

問題	原因	解決策
他人のコメントも編集できる	権限チェックがない	<code>session.user.id === comment.userId</code> で判定
編集中に削除ボタンが表示される	条件式に <code>editingId</code> を含めていない	<code>!editingId</code> を追加
キャンセル後に内容が残る	<code>state</code> をクリアしていない	<code>setEditContent("")</code> を実行

次回予告

Day 20では、タスクの検索機能を実装します。